Fundamentals of Machine Learning

Tutorial 2: Learning by Combining Models

Central question
How can several models be combined so that their different errors improve prediction, while controlling complexity and evaluating the resulting ensemble honestly?

Explain your reasoning and state any assumptions you make.

Problems

1. **Hard voting, soft voting, and confidence**

   Three classifiers predict the probability of an attack for four flows:

Flow	p_A	p_B	p_C	y
1	0.49	0.49	0.95	1
2	0.80	0.55	0.05	1
3	0.40	0.40	0.90	0
4	0.10	0.70	0.70	1

   Each individual classifier uses threshold 0.5.

   (a) Find the hard-voting and soft-voting prediction for each flow. Whenever they disagree, explain what information one rule uses that the other discards.

   (b) Suppose classifier C is often too confident. Explain how this affects hard and soft voting. Does this mean that hard voting is always better? Give a counterexample.

2. **Why can weaker trees form a stronger forest?**

   A dataset has $d = 25$ features. Feature x_1 is highly predictive, so many bagged trees choose it at the root. A random forest offers only $m_{\text{try}} = 5$ randomly selected features at each split.

   (a) Derive the probability that x_1 is available at one split and the probability that it is unavailable at the roots of all $M = 10$ trees.

   (b) Explain why hiding x_1 from many roots may reduce the accuracy of individual trees but improve the forest.

   (c) Explain how you would choose m_{try} and tree size for the whole forest. Why is it inappropriate to tune each tree separately?

3. **Interpreting feature importance**

For a node t, define its MDI credit as

$$\Delta_t = \frac{N_t}{N}\left[I(S_t) - \frac{N_L}{N_t}I(S_L) - \frac{N_R}{N_t}I(S_R)\right].$$

A root containing all N observations reduces Gini impurity from 0.50 to 0.45. A deeper node containing $0.30N$ observations reduces its impurity from 0.48 to 0.08.

(a) Compute the MDI credit of each split. Why can the deeper split receive more credit even though it contains fewer observations?

(b) A fitted forest has validation error 0.12. After permuting x_a, the error is 0.20; after permuting x_b, it is 0.14. Compute the permutation importance of each feature. What do the two values say about how much the fitted forest relies on these features?

(c) Suppose x_a and x_b are strongly correlated and carry similar information. Explain why shuffling only one of them may make it appear less important than it really is. Why does high feature importance not prove that the feature causes the outcome?

4. **Fitting residuals with shrinkage**

Consider four regression observations:

x	1	2	3	4
y	0	2	4	6

Use squared loss and initialize with $F_0 = \overline{y}$. A depth-one tree splits at $x = 2.5$.

(a) Compute the initial residuals and the mean residual in each leaf. Write the fitted correction tree $T_1(x)$.

(b) With learning rate $\eta = 0.5$, compute F_1 and the new residuals. Compare the squared-error sums before and after the update.

(c) When $\eta < 1$, the first tree corrects only part of the residuals. Explain why leaving some error for later trees can improve validation performance.

5. **From pseudo-residuals to Newton leaf values**

For binary cross-entropy, let $F^{(i)}$ be the current logit and $p^{(i)} = \sigma(F^{(i)})$.

(a) Starting from $\ell(y, F) = -y \log p - (1 - y)\log(1 - p)$, prove that

$$g^{(i)} = \frac{\partial \ell^{(i)}}{\partial F^{(i)}} = p^{(i)} - y^{(i)}, \qquad h^{(i)} = \frac{\partial^2 \ell^{(i)}}{\partial (F^{(i)})^2} = p^{(i)}(1 - p^{(i)}).$$

(b) A proposed leaf contains three observations:

$$(y^{(i)}, p^{(i)}) = (1, 0.2), (1, 0.4), (0, 0.8).$$

Compute $G = \sum_i g^{(i)}$, $H = \sum_i h^{(i)}$, and the Newton leaf value both without regularization and with $\lambda = 1$. If $\eta = 0.1$, what correction is actually added to the logit?

(c) For one observation, let p be very close to zero. Its Newton pseudo-target $-g/h$ can then be very large. What happens to its weight h at the same time, and why must the target and weight be considered together?

6. **A split must pay for its extra leaf**

An XGBoost parent node has $(G, H) = (-4, 3)$. A candidate split produces left child $(G_L, H_L) = (-3, 1)$ and right child $(G_R, H_R) = (-1, 2)$. Let $\lambda = 1$.

(a) For a leaf with derivative sums (G, H), its regularized quadratic contribution is

$$\phi(w) = Gw + \frac{1}{2}(H + \lambda)w^2.$$

Find the value w^* that minimizes $\phi(w)$, then substitute it into ϕ to obtain the minimum $\phi(w^*)$. Use these minimum values to calculate how much the objective decreases when the parent is replaced by the two children. After subtracting $\gamma = 0.2$, should XGBoost accept the split?

(b) Find the largest value of γ for which the split is accepted.

(c) A binary split replaces one leaf with two. Use the penalty γQ to explain why this costs exactly one γ. How does this effect differ from increasing λ?

7. **Choosing a model from validation results**

Five gradient-boosting configurations produce the following results. The selected round is the round with minimum validation log loss.

Configuration	η	depth	selected round	training loss	validation loss	fit time (s)
A	0.10	1	80	0.31	0.34	12
B	0.10	3	110	0.12	0.29	28
C	0.10	5	70	0.05	0.35	42
D	0.03	3	340	0.14	0.27	75
E	0.03	5	240	0.04	0.30	110

(a) If the selection criterion is validation log loss, which configuration should be chosen? Why would choosing by training loss give a different answer?

(b) Which configurations appear too complex for these data? What plot would provide stronger evidence of overfitting?

(c) If training must finish within 40 seconds, which configuration would you recommend? State the loss in predictive performance caused by this time limit.

(d) Explain why the smaller learning rate generally selects more rounds. Does this table prove that a smaller learning rate is always better?

(e) After choosing the configuration, explain how to use the training, validation, and test sets without allowing the test results to influence any modeling decision.

8. **Avoiding leakage in an ensemble experiment**

A team compares a decision tree, random forest, gradient boosting model, and XGBoost. After viewing the test results, it repeatedly changes the hyperparameters. For stacking, it trains the meta-model using predictions that the base models made on their own training observations.

(a) Identify the two ways in which this procedure gives results that are too optimistic. Then describe a correct training, model-selection, and test procedure.

(b) What is the difference between OOB predictions and out-of-fold predictions? Describe how to produce the out-of-fold predictions needed to train the stacking meta-model.

(c) Once every modeling choice has been fixed, the team considers refitting the base models using both the training and validation data. When is this reasonable, and how can the test set still provide a fair final evaluation?

9. **Deployment when the data may change**

A security team finds that its deep decision tree changes substantially when the training sample changes. A random forest is more stable, but it relies heavily on destination port. The validation data contain the same services as the training data, while the deployed system will also encounter new services and ports.

(a) Explain what bagging, a random forest, and boosting could each help with in this situation. Design an experiment that separates improvement due to the ensemble from improvement due to using destination port.

(b) What validation results would make you trust the chosen model on the known services? Why would these results still not establish performance on new services?

(c) The team proposes stacking all the models to handle the new services. Explain why stacking may not solve this problem. Suggest a change to the data split or evaluation design that would provide more useful evidence.